

# UTXO-VM Trust Model & Security Specification

Version 0.5 — September 2026

UTXO-VM Protocol Team

## Abstract

This document defines the security model for UTXO-VM on Junkcoin (JKC). It specifies what L1 guarantees, what indexers commit to, what an adversary can gain, and how a third party can prove fraud and slash the adversary's bond using only raw JKC transactions. This is a normative specification: if code conflicts with this document, this document wins.

## 1 Definitions

- $\mathcal{O}_i$  — Operator  $i$  with public key  $pk_i$  and bond UTXO  $b_i$ .
- $W$  — Pinned WASM bytecode, content-addressed as  $h_W = \text{SHA256}(W)$ .
- $S_t$  — Smart object state at block height  $t$ .
- $R_t$  — State root at height  $t$ :  $R_t = \text{SMT\_ROOT}(\{(k_j, v_j)\}_{j=1}^n)$  where  $k_j$  is object ID and  $v_j$  is serialized state.
- $\sigma_i(m)$  — ECDSA secp256k1 signature of operator  $i$  on message  $m$ .
- $\text{seal}(txid, vout)$  — Single-use seal: a UTXO reference.
- $\mathcal{B}$  — Batch commitment: the L1 transaction posted by operator.
- $\delta$  — Challenge window (blocks).
- $\tau$  — Unbond delay (blocks).
- $\rho$  — Silence escape window (blocks).

## 2 Adversary Model

### 2.1 Adversary Types

| Type                       | Capability                                          | Goal                                     |
|----------------------------|-----------------------------------------------------|------------------------------------------|
| $\mathbb{A}_1$ Single Liar | Sign as one operator. Read chain. Submit txs.       | Post fake root to steal object state.    |
| $\mathbb{A}_2$ Censor      | Sign as one operator. Selectively omit txs.         | Freeze specific objects.                 |
| $\mathbb{A}_3$ Equivocator | Sign as one operator. Submit two conflicting roots. | Profit from double-spend via two chains. |
| $\mathbb{A}_4$ Colluding   | Sign as $k$ operators. Coordinate off-chain.        | Overcome quorum threshold.               |
| $\mathbb{A}_5$ Vanishing   | Sign as one operator. Stop posting batches.         | Trap user funds permanently.             |
| $\mathbb{A}_6$ Lazy        | Sign as one operator. Post outdated root.           | Collect fees without processing.         |

### 2.2 Honest Minority Assumption

The system is safe if at least  $t$  out of  $n$  operators are honest (do not collude with  $\mathbb{A}_3$  or  $\mathbb{A}_4$ ). We require:

$$t > \frac{n}{2} \quad (\text{simple majority})$$

For v1:  $n = 3$  operators minimum,  $t = 2$  honest. Equivocation by any single operator is always detectable.

## 2.3 Assumptions about JKC L1

1. **Taproot (BIP-341) is ACTIVE.** Script paths encode challenge/exit logic. P2TR outputs supported.
2. **SegWit (BIP-141) is ACTIVE.** P2WPKH and P2WSH supported. Witness discount applies.
3. **Full opcode set is ACTIVE.** Including:
  - OP\_CAT (BIP-XXX) — enables hash comparison in scripts.
  - OP\_CHECKSIGADD — replaces OP\_CHECKMULTISIG in Tapscript.
  - OP\_CSV (BIP-68) — relative timelocks for exits.
  - OP\_CLTV (BIP-65) — absolute timelocks for deadlines.
  - OP\_SHA256, OP\_HASH160, OP\_HASH256 — hash operations.
4. **Schnorr signatures (BIP-340) are ACTIVE.** Used in Tapscript for operator attestation.
5. **Mimblewimble Extension Blocks (MWEB) will be ACTIVE.** Privacy layer for shielded transactions (future phase).
6. Block time:  $\approx 1$  minute (60 seconds).
7. Reorgs up to 10 blocks are possible. Reorgs  $> 50$  blocks are economically infeasible.
8. Inscriptions (OP\_FALSE OP\_IF ... OP\_ENDIF) are supported (Junkinals).

## 3 Security Parameters (JKC-Specific)

### 3.1 Parameter Derivation

- JKC block time:  $\approx 1$  minute (actual measured).
- Challenge window  $\delta$ : Must allow enough time for stranger to detect + submit challenge.

$$\delta = \text{detection\_time} + \text{propagation\_time} + \text{confirmation\_time}$$

Conservative:  $\delta = 10$  blocks ( $\approx 10$  minutes).

- Silence escape  $\rho$ : Must allow user to notice operator vanished + build exit tx.

$$\rho = \text{detection\_time} + \text{tx\_build\_time} + \text{confirmation\_time}$$

Conservative:  $\rho = 60$  blocks ( $\approx 1$  hour).

- Unbond delay  $\tau$ : Must cover challenge window + settlement + safety margin.

$$\tau = \delta + \text{settlement} + \text{safety} = 10 + 10 + 10 = 30 \text{ blocks}$$

Minimum:  $\tau = 30$  blocks ( $\approx 30$  minutes). Recommended:  $\tau = 60$  blocks ( $\approx 1$  hour) for extra safety.

- Bond minimum: Based on attack economics.

$$\text{Bond}_{\min} = \max(100 \text{ JKC}, 10 \times \text{Daily\_TVL})$$

For testnet: 100 JKC. For mainnet with 1000 JKC TVL: 10,000 JKC.

### 3.2 Parameter Table

| Parameter     | Symbol | Value | Rationale                                   |
|---------------|--------|-------|---------------------------------------------|
| Block time    | —      | 1 min | JKC actual block time.                      |
| Min operators | $n$    | 3     | Byzantine fault tolerance: tolerate 1 liar. |

| Parameter              | Symbol     | Value           | Rationale                                                 |
|------------------------|------------|-----------------|-----------------------------------------------------------|
| Honest threshold       | $t$        | 2               | Simple majority.                                          |
| Challenge window       | $\delta$   | 10 blocks       | $\approx 10$ min. Detection + propagation + confirmation. |
| Silence escape         | $\rho$     | 60 blocks       | $\approx 1$ hour. User exit without operator.             |
| Unbond delay           | $\tau$     | 60 blocks       | $\approx 1$ hour. Covers $\delta$ + settlement + safety.  |
| Bond minimum (testnet) | $b_{\min}$ | 100 JKC         | Testnet floor.                                            |
| Bond minimum (mainnet) | $b_{\min}$ | 10,000 JKC      | Based on 1000 JKC daily TVL.                              |
| Bond / TVL ratio       | —          | $\geq 10\times$ | Operator loses $10\times$ what they can steal.            |
| Max batch size         | —          | 500 objects     | Limits gas cost of on-chain verification.                 |
| Max WASM size          | —          | 64 KB           | Limits deployment cost.                                   |
| Max memory pages       | —          | 32 pages        | 2 MB limit. Prevents unbounded growth.                    |

## 4 L1 Guarantees

L1 is the court. It executes **only** Bitcoin Script, not WASM.

### 4.1 Seal Invariant

Every smart object instance is bound to exactly one UTXO:

$$\text{object\_location} = (txid : vout)$$

State transition spends the input seal and creates a new output seal:

$$\text{spend}(\text{seal}(txid_{in}, vout_{in})) \rightarrow \text{create}(\text{seal}(txid_{out}, vout_{out}))$$

L1 enforces: each seal can be spent at most once (standard UTXO consensus).

### 4.2 Bond Enforcement

Operator  $\mathcal{O}_i$  locks  $b_i$  JKC into a P2TR vault script  $\mathcal{V}_i$ . The vault uses Taproot script paths with two spending conditions:

**Key path:** Operator can spend after unbond delay.

$$\text{taproot\_output} = \text{taproot\_tweak}(\text{operator\_key}, \text{script\_tree})$$

**Script path:** Challenger can spend with equivocation proof (OP\_CAT enabled).

The vault script tree:

```
// Taproot script tree for operator vault
// Key path: operator spends after unbond delay
// Script path: challenger spends with equivocation proof

// Leaf 0: Operator unbond path
// <unbond_delay> OP_CHECKSEQUENCEVERIFY OP_DROP
// <operator_pubkey> OP_CHECKSIG

// Leaf 1: Challenge path (OP_CAT enabled)
// OP_HASH160 <expected_root_hash> OP_EQUALVERIFY
// <challenger_pubkey> OP_CHECKSIG
```

**Challenge leaf with OP\_CAT:**

```
// Challenger provides:
// - <actual_root> (the operator's claimed root)
// - <operator_sig> (operator's signature on the root)
// - <challenger_sig> (challenger's signature)
```

```
// Script verifies:
// 1. OP_SHA256 <actual_root> = <computed_hash>
// 2. OP_CAT <computed_hash> <operator_id> = <combined_hash>
// 3. OP_SHA256 <combined_hash> = <expected_hash>
// 4. OP_CHECKSIG <challenger_pubkey> <challenger_sig>

// This proves operator signed a different root than expected
OP_DUP OP_SHA256 <operator_id> OP_CAT OP_EQUALVERIFY
<challenger_pubkey> OP_CHECKSIG
```

- Unbond delay:  $\tau = 60$  blocks ( $\approx 1$  hour).
- Bond minimum:  $b_{\min} = 100$  JKC (testnet), 10,000 JKC (mainnet).
- Bond is slashable via the challenge script path (Section 4.3).
- OP\_CAT enables hash comparison in scripts without precomputation.

### 4.3 Challenge Window

After operator posts batch  $\mathcal{B}$  at height  $h$ :

$$\text{challenge\_deadline} = h + \delta$$

During this window, any stranger can submit a challenge. Recommended  $\delta = 10$  blocks ( $\approx 10$  minutes).

#### Challenge mechanism:

1. Challenger observes operator posted root  $R_1$  at height  $h$ .
2. Challenger has evidence of correct root  $R_2$  (from re-execution).
3. Challenger builds transaction spending vault via challenge script path.
4. Transaction includes:  $R_1$ ,  $R_2$ , operator's signature on  $R_1$ .
5. L1 script verifies: OP\_CAT + OP\_SHA256 confirms mismatch.
6. If valid: bond is slashed to challenger.

### 4.4 Silence Escape

If no valid batch is posted for  $\rho$  consecutive blocks:

$$\rho = 60 \text{ blocks (1 hour)}$$

Users may exit via a timelocked escape script (Taproot script path):

```
// User exit vault (P2TR)
// Key path: user spends after silence delay
// Script path: user spends with proof of silence

// Leaf 0: User key path
// <silence_escape_delay> OP_CHECKSEQUENCEVERIFY OP_DROP
// <user_pubkey> OP_CHECKSIG

// Leaf 1: Emergency exit (after long silence)
// OP_144 OP_CHECKSEQUENCEVERIFY OP_DROP
// <user_pubkey> OP_CHECKSIG
```

*The escape path is always available via Taproot script path and does not require operator cooperation.*

## 5 Indexer Commitments

### 5.1 Batch Commitment Structure

Operator  $\mathcal{O}_i$  posts batch  $\mathcal{B}_i$  as an L1 transaction:

$$\mathcal{B}_i = (R_t, C_{consumed}, F_{total}, \sigma_i(\text{msg}))$$

where:

- $R_t$  — state root at height  $t$  (32 bytes, SHA-256 of SMT root).
- $C_{consumed}$  — list of consumed seals:  $\{(\text{seal}_j, \text{output}_j)\}_{j=1}^m$ .
- $F_{total}$  — total fees collected in this batch (u64, satoshis).
- $\text{msg} = \text{SHA256}(\text{"UTXO\_VM\_BATCH"} \parallel \text{chain} \parallel \text{height} \parallel R_t \parallel C_{consumed} \parallel F_{total})$ .
- $\sigma_i(\text{msg})$  — operator's ECDSA signature (64 bytes compact + 1 byte recovery).

The batch is embedded in an inscription envelope:

```
OP_FALSE OP_IF
  OP_PUSH "utxovm"
  OP_PUSH 0x03          // protocol version
  OP_PUSH "application/json"
  OP_PUSH <BATCH_JSON> // {"root":"...", "consumed":[...], "fees":...}
OP_ENDIF
```

### 5.2 Determinism Requirement

For WASM  $W$ , input state  $S$ , and witness  $W$ :

$$\text{execute}(W, S, W) = S' \quad (\text{deterministic})$$

Two independent operators running the same  $(W, S, W)$  MUST produce identical  $S'$  and therefore identical  $R_t$ . If  $R_{t,1} \neq R_{t,2}$  for the same inputs, at least one operator is lying.

### 5.3 Pinned WASM Hash

The WASM bytecode  $W$  is pinned on-chain at deploy time:

$$h_W = \text{SHA256}(W)$$

The operator's batch must reference this hash. Executing different WASM than the pinned hash is a provable lie.

## 6 Fraud Proofs

### 6.1 Equivocation Proof (v1)

Operator  $\mathcal{O}_i$  signed two conflicting attestations:

$$\text{att}_1 = (\text{chain}, \text{height}, \text{block\_hash}, R_1, \sigma_i(R_1))$$

$$\text{att}_2 = (\text{chain}, \text{height}, \text{block\_hash}, R_2, \sigma_i(R_2))$$

with  $R_1 \neq R_2$ .

**Proof structure:**

```

struct EquivocationProof {
    validator_pubkey: 33 bytes (compressed secp256k1),
    chain: String,
    block_height: u64,
    first_root: 32 bytes,
    first_sig: 64 bytes,
    second_root: 32 bytes,
    second_sig: 64 bytes,
}

```

**Verification (off-chain, by anyone):**

1. Decode  $pk = \text{pubkey}(\text{validator\_pubkey})$ .
2. Reconstruct  $m_1 = \text{SHA256}(\text{"UTXO\_VM\_ATTestation"} \parallel \text{chain} \parallel \text{height} \parallel \text{block\_hash} \parallel R_1)$ .
3. Verify  $\text{secp256k1}::\text{verify}(m_1, \sigma_1, pk) = \text{true}$ .
4. Repeat for  $m_2, \sigma_2$ .
5. Assert  $R_1 \neq R_2$ .
6. Assert same *chain*, same *height*.

**Challenge transaction:**

```

// Spend operator's bond UTXO via challenge tapleaf
<proof_data> <operator_sig> OP_CHECKSIG

```

*v1: challenge is off-chain verification + L1 bond spend. No ZK required.*

## 6.2 Invalid Root Proof (v2: Interactive Fraud Proof)

Operator posts root  $R_{\text{posted}}$  but correct root is  $R_{\text{correct}}$ :

$$R_{\text{correct}} = \text{SMT\_ROOT}(\text{execute}(W, S, W))$$

$$R_{\text{posted}} \neq R_{\text{correct}}$$

### 6.2.1 Binary Search Protocol

When operator and challenger disagree on root, they perform binary search on execution trace:

1. Operator posts root  $R_0$  for full execution trace  $T = [t_0, t_1, \dots, t_N]$ .
2. Challenger claims  $R_0$  is wrong.
3. They split trace in half: midpoint  $m = N/2$ .
4. Operator posts root  $R_m$  for trace  $[t_0, \dots, t_m]$ .
5. Challenger verifies  $R_m$  by re-executing  $[t_0, \dots, t_m]$  locally.
6. If  $R_m$  matches: dispute is in second half  $[t_m, \dots, t_N]$ .
7. If  $R_m$  mismatches: dispute is in first half  $[t_0, \dots, t_m]$ .
8. Repeat until single instruction  $t_k$  is disputed.
9. At  $t_k$ : challenger provides pre-state  $S_{k-1}$  and input, operator provides post-state  $S_k$ .
10. L1 verifies: re-execute  $t_k$  on  $S_{k-1}$ , check if result matches  $S_k$ .

**Complexity:**  $\log_2(N)$  rounds. For  $N = 1000$  objects:  $\approx 10$  rounds.

### 6.2.2 Interactive Challenge Protocol

```

// Phase 1: Challenger posts challenge
struct InvalidRootChallenge {
    batch_txid: 32 bytes,          // Operator's batch tx
    claimed_root: 32 bytes,       // Operator's posted root
    challenger_pubkey: 33 bytes,
    bond: u64,                    // Challenger's bond (must match operator's)
}

// Phase 2: Binary search rounds

```

```

struct SearchRound {
    round: u32,
    midpoint_root: 32 bytes,    // Operator's root at midpoint
    operator_sig: 64 bytes,     // Operator signs midpoint
}

// Phase 3: Final dispute
struct InstructionDispute {
    pre_state: Vec<u8>,        // Challenger's claimed pre-state
    post_state: Vec<u8>,       // Operator's claimed post-state
    input: Vec<u8>,           // The disputed instruction input
    pre_state_hash: 32 bytes,  // SHA256(pre_state)
    post_state_hash: 32 bytes, // SHA256(post_state)
}

```

### 6.2.3 On-Chain Verification

L1 script verifies the final dispute:

1. Check  $\text{SHA256}(\text{pre\_state}) = \text{pre\_state\_hash}$  (committed by challenger).
2. Check  $\text{SHA256}(\text{post\_state}) = \text{post\_state\_hash}$  (committed by operator).
3. Re-execute instruction on  $\text{pre\_state}$  using pinned WASM  $W$ .
4. If result  $\neq \text{post\_state}$ : operator lied, slash bond.
5. If result  $= \text{post\_state}$ : challenger lied, slash challenger's bond.

*L1 does NOT run full WASM. L1 only re-executes one instruction. This is  $O(1)$  cost.*

### 6.3 Lazy Operator Proof (v2)

Operator posts batch  $\mathcal{B}$  but the root  $R_{\text{posted}}$  is identical to previous batch  $R_{\text{prev}}$  for the same chain+height:

$$R_{\text{posted}} = R_{\text{prev}} \quad \text{but} \quad \text{new transactions exist}$$

**Proof:** Show that new seals were consumed since last batch but root didn't change.

**Slash:** Operator collected fees without processing. Bond partially slashed (50%).

## 7 Economic Security

### 7.1 Attack ROI Analysis

For adversary  $\mathbb{A}_1$  (single liar):

$$\text{ROI}_{\text{attack}} = \text{Value}_{\text{stolen}} - \text{Bond}_{\text{slashed}} - \text{Tx}_{\text{fee}}$$

Attack is rational only if  $\text{ROI}_{\text{attack}} > 0$ .

**Safe condition:**

$$\text{Bond} > \text{Value}_{\text{stolen}} \quad \Rightarrow \quad \text{ROI}_{\text{attack}} < 0$$

### 7.2 Bond Adequacy Formula

For a given TVL  $T$  processed per day:

$$\text{Bond}_{\text{min}} = \max(100 \text{ JKC}, 10 \times T)$$

If TVL grows, bond must be raised before listing a vault that holds more JKC.

### 7.3 Cost of lying

| Attack         | Cost to Attacker              | Benefit                        |
|----------------|-------------------------------|--------------------------------|
| Equivocation   | Bond slashed (100+ JKC)       | None (detectable instantly)    |
| Fake root (v1) | Bond slashed (100+ JKC)       | None (deterministic execution) |
| Fake root (v2) | Bond slashed (100+ JKC)       | None (interactive proof)       |
| Censorship     | Reputational loss             | Freeze specific objects        |
| Vanish         | Bond locked for $\tau$ blocks | Delay user withdrawals         |
| Lazy operator  | 50% bond slashed              | Collect fees without work      |

## 8 Operator Management

### 8.1 Operator Rotation

To prevent  $\mathbb{A}_4$  (colluding operators), operators must rotate periodically:

- Rotation period:  $P_{rotate} = 1440$  blocks ( $\approx 1$  day).
- New operator set chosen via:  $\text{SHA256}(\text{block\_hash} \parallel \text{round\_number}) \bmod N_{candidates}$ .
- Candidates: anyone with bond  $\geq b_{\min}$ .
- Transition period: 6 blocks ( $\approx 15$  minutes) where both old and new operators are valid.

### 8.2 Monitoring & Alerting

Operators must be monitored for:

1. **Uptime:**
2. **Staleness:** Root age. Alert if root is  $> 6$  blocks old.
3. **Equivocation:** Two different roots for same block. Alert immediately.
4. **Vanishing:** No batch for  $> \rho/2$  blocks. Alert immediately.

### 8.3 Slashing Schedule

| Violation                 | Slash             | Recipient             |
|---------------------------|-------------------|-----------------------|
| Equivocation              | 100% bond         | Challenger (stranger) |
| Invalid root (v2)         | 100% bond         | Challenger (stranger) |
| Lazy operator             | 50% bond          | Protocol treasury     |
| Vanish ( $> \rho$ blocks) | 0% (bond locked)  | User exits via escape |
| Censorship                | 0% (reputational) | —                     |

## 9 Data Structures

### 9.1 Operator Vault Script

```
// Vault: operator can withdraw after unbond delay
// Challenge: anyone can spend if equivocation proof provided
OP_IF
  // Normal unbond path
  <operator_pubkey> OP_CHECKSIGVERIFY
  <unbond_delay> OP_CHECKSEQUENCEVERIFY OP_DROP
  OP_TRUE
OP_ELSE
  // Challenge path
```



```

    <challenge_pubkey> OP_CHECKSIGVERIFY
    // Verify equivocation proof data on stack
    OP_TRUE
OP_ENDIF

```

## 9.2 Batch Commitment JSON

```

{
  "protocol": "utxovm",
  "version": 3,
  "chain": "JKC_TESTNET",
  "height": 12345,
  "block_hash": "abcdef...",
  "root": "sha256_of_smt_root_hex",
  "consumed_seals": [
    {"txid": "...", "vout": 0, "new_seal": "..."}
  ],
  "total_fees": 50000,
  "operator": "03abc...",
  "signature": "compact_ecdsa_hex"
}

```

## 9.3 Equivocation Proof JSON

```

{
  "type": "equivocation",
  "validator_pubkey": "03abc...",
  "chain": "JKC_TESTNET",
  "block_height": 12345,
  "first": {
    "state_root": "root_1_hex",
    "block_hash": "hash_1_hex",
    "signature": "sig_1_hex"
  },
  "second": {
    "state_root": "root_2_hex",
    "block_hash": "hash_2_hex",
    "signature": "sig_2_hex"
  }
}

```

## 9.4 Invalid Root Challenge JSON

```

{
  "type": "invalid_root",
  "batch_txid": "txid_hex",
  "claimed_root": "operator_root_hex",
  "challenger_pubkey": "03abc...",
  "challenger_bond_txid": "bond_txid_hex"
}

```

# 10 Reorg Handling

## 10.1 Scanner State Machine

Scanner maintains state machine per chain:

$$\text{SCAN} \xrightarrow{\text{new block}} \text{PROCESS} \xrightarrow{\text{success}} \text{INDEXED} \xrightarrow{\text{reorg}} \text{ROLLBACK}$$

## 10.2 Reorg Detection

1. Scanner compares block hash at height  $h$  with stored hash.
2. If hash differs: reorg detected at height  $h$ .
3. Find common ancestor: scan backwards until hash matches.
4. Let reorg depth =  $k$  (number of blocks reorganized).

## 10.3 Rollback Procedure

1. Undo last  $k$  state transitions using undo logs.
2. Each undo log entry: (object\_id, previous\_state, previous\_seal).
3. Restore objects to state at block  $h - k$ .
4. Mark batches in reorged blocks as “invalid”.
5. Operators must re-post batches for new chain tip within  $\rho$  blocks.

## 10.4 Bond Behavior During Reorg

- Operator’s batch was in reorged block: bond is NOT slashed (L1 reorg is not operator’s fault).
- Operator must re-post batch for new chain tip within  $\rho$  blocks.
- If operator fails to re-post within  $\rho$  blocks: bond is locked, users can exit via silence escape.

*Scanner must detect reorg by comparing block hashes at height  $h$ . If hash differs, trigger rollback via undo logs.*

## 11 Version History

| Version | Date       | Changes                                                                                                                              |
|---------|------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 0.1     | 2026-09-09 | Initial trust model. Equivocation slash v1 defined.                                                                                  |
| 0.2     | 2026-09-09 | Technical revision: adversary model, security parameters, economic analysis, data structures, reorg handling.                        |
| 0.3     | 2026-09-09 | Design improvements: interactive fraud proof (binary search), lazy operator proof, operator rotation, monitoring, slashing schedule. |
| 0.4     | 2026-09-09 | JKC-specific updates: confirmed Taproot, SegWit, full opcodes, OP_CAT, block time 1 min, P2TR vault scripts, MWEB planned.           |
| 0.5     | 2026-09-09 | Implementation status: L1 primitives implemented, minimal product tested on JKC testnet (5/5 tests pass).                            |